Marvel Timelines: the story of how it was built

Everything important behind marvel.harver.in, told as a story with diagrams, and mapped to the skills software engineers use: web development, DevOps, networking, security, system design (HLD and LLD) and SEO. Updated 30 October 2026.

Each chapter carries tags for the skills it shows: WEB DEV DEVOPS NETWORKING SECURITY HLD / LLD SEO

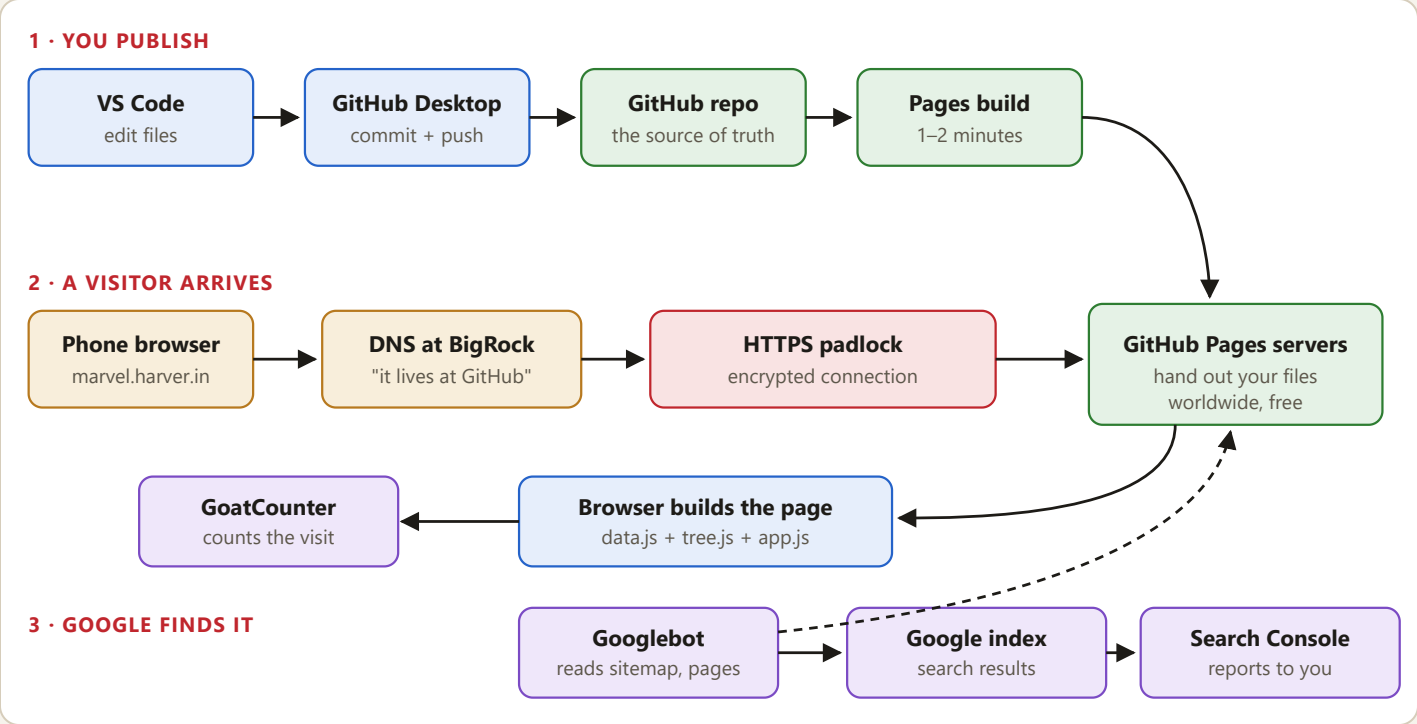
Chapters

1. The whole map on one page
2. Building the pages (web dev, LLD)
3. Shipping changes safely (DevOps)
4. Giving it a name (DNS)
5. Locking the door (security)
6. Why it's built this way (HLD)
7. Getting found (SEO and AI search)
8. Watching who comes (analytics)
9. Bugs we hit and what they taught
10. Everyday tasks, step by step
11. Getting traffic
12. Which file does what
13. Words to know

CHAPTER 1

The whole map on one page

> It started as one X-Men timeline chart. It grew into nine pages covering 169 Marvel films, series and cartoons, a domain of its own, visitor stats, and search listings. Here is every moving part and how they connect. The rest of this guide zooms into one part at a time.



Three journeys meet at GitHub's servers: yours (publishing), a visitor's (reading), and Google's (indexing).

The one-line summary. It's a *static site*: plain files with no server program and no database. GitHub stores and serves the files, BigRock points your name at GitHub, and the visitor's own browser does the work of building each page.

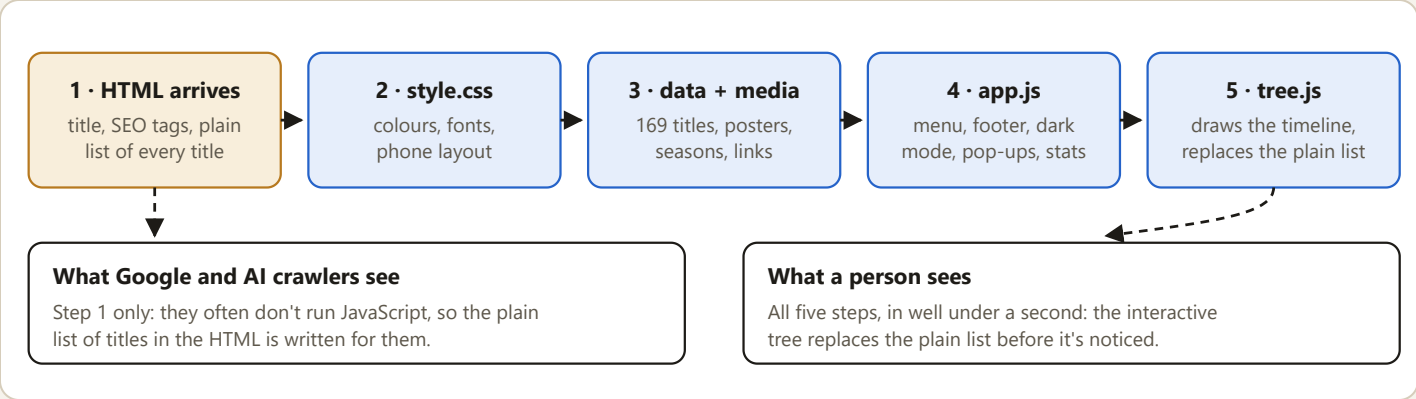
CHAPTER 2

Building the pages

WEB DEV LLD

> Typing 169 titles into nine HTML pages by hand would mean fixing every typo in many places. So the titles live in one data file, and small scripts turn that data into trees, cards and pop-ups inside the visitor's browser. Change one line of data, and every page that shows that title updates.

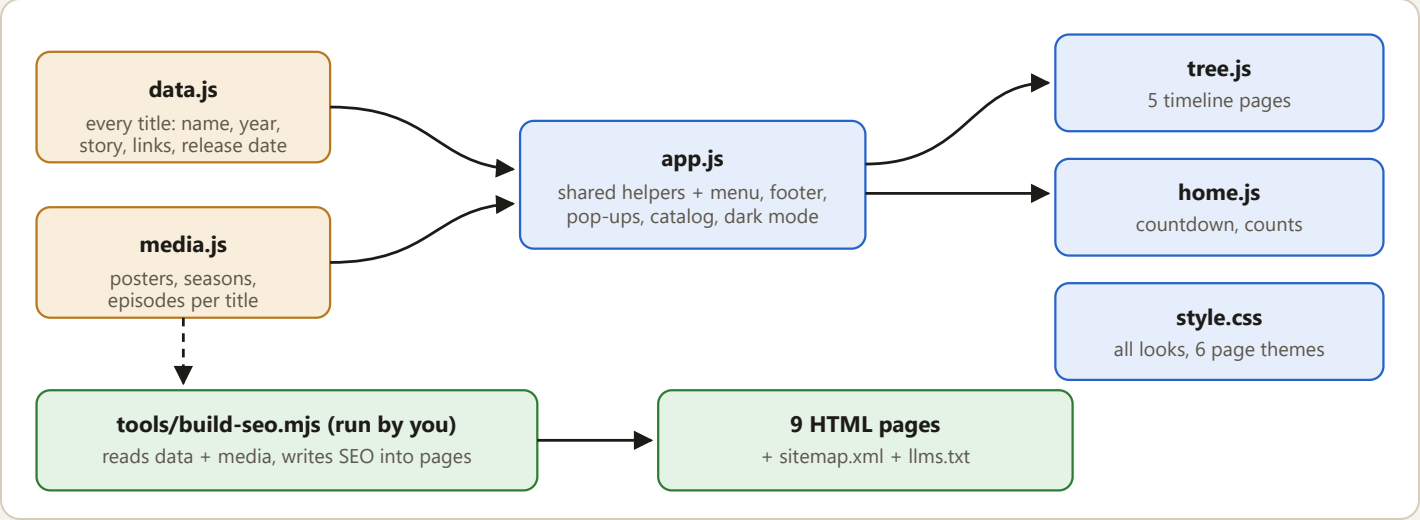
What happens when a page opens



This is called **progressive enhancement**: a page that works as plain text, then gets richer when JavaScript runs.

How the files depend on each other (low-level design)

Low-level design (LLD) is about the pieces inside one system: what each module does and who uses whom. Each file here has one job:



Data flows one way: data files → shared code → page-specific code. That makes changes predictable. This is **separation of concerns**, and **a single source of truth** (data.js).

Front-end ideas used on the site

Idea	Where it is	In plain words
Responsive design	`@media (max-width:820px)` in style.css	Different rules for phone widths: one lane per block, smaller menus, smaller buttons.
CSS variables (design tokens)	`--bg`, `--ink`, `--accent` per `data-theme`	Each page sets its own colour set: MCU red, X-Men steel, Street Level blood red, Spider-Verse blue/red, 2000s orange, Multiverse dark Loki/Doom green. One rule, many looks.
Dark theme	moon button in app.js, small script in each page head	Follows the phone's setting until the visitor picks; the choice is saved in the browser (`localStorage`). The head script applies it before the page draws, so there's no white flash.
Hide-on-scroll bar	tree.js	Scrolling down hides the section bar for more room; any scroll up brings it back.

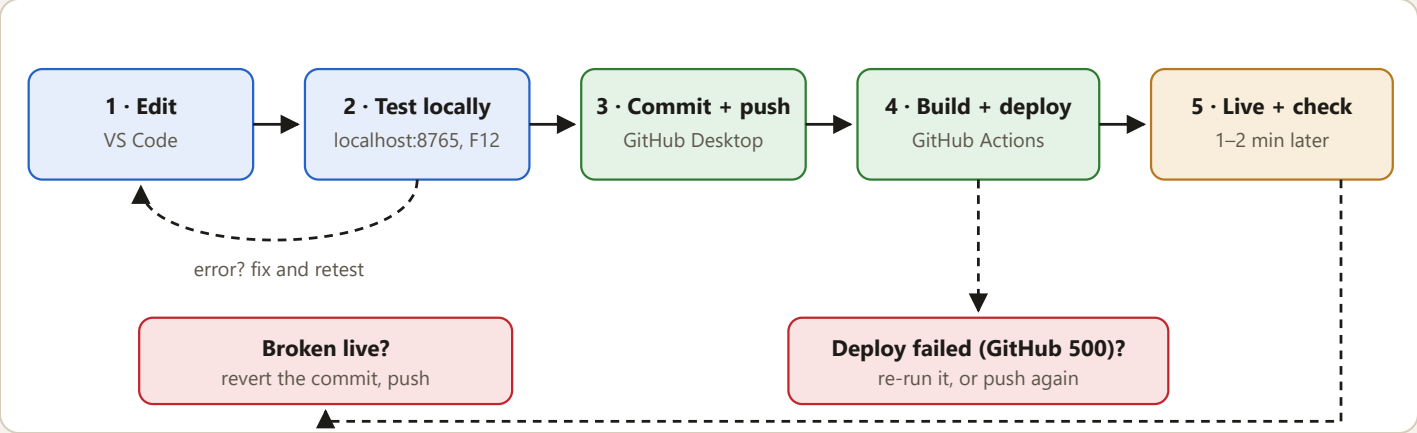
Filter by character	`CHARACTERS` in data.js, catalog code in app.js	The Catalog's character list (33 heroes and villains) isn't typed title by title. Each character has a **rule**: a pattern of names to look for in the title and summary (Iron Man, Tony Stark), plus an `add` list for appearances the summary doesn't mention and a `not` list for false hits. New titles join the right characters by themselves.
Feedback form	"Send feedback" in every footer, code in app.js	A pop-up asks how useful the site is, a topic, a message and an optional email. A static site has no server to receive it, so a free service, **Web3Forms**, takes the message and emails it to you. Until its key is added, the button opens the visitor's email app with the message filled in.
Icon links	Contact icons in the footer	GitHub, LinkedIn, Instagram and email are drawn as small inline SVG icons, so they stay sharp and follow the page colour. Each has an `aria-label` and a hover label so screen readers and visitors know what it opens.
State in the address	catalog filters, `#id` links	A link like `mcu.html#iw` opens a specific title, so links can be shared.
Accessibility	`aria-pressed`, `alt` text, `aria-label`	Screen readers can tell which buttons are on, what each image shows, and what icon buttons do.
Self-updating content	`rel` dates in data.js, `FILMS` in home.js	Upcoming titles lose their tag on release day; the countdown moves to the next film on its own.

CHAPTER 3

Shipping changes safely

DEVOPS

> Hundreds of changes went live during this project, and the site was never left broken. That came from a fixed routine: test on your own computer, push, let GitHub deploy, then check. DevOps is exactly this: getting changes from a laptop to real users quickly without breaking things.



A deploy pipeline with a safety net at each stage. Every push is saved forever in git history, so any version can be brought back.

The tools in that pipeline

Step	Tool	What to know
Test locally	`python -m http.server 8765` in the project folder	Then open `http://localhost:8765`. Press F12 → Console for errors, and the phone icon for phone width.
Version control	git, via GitHub Desktop	A **commit** is a saved snapshot with a message. **Push** uploads it. **Fetch/Pull** brings down changes made on GitHub (like the CNAME file GitHub created).
CI/CD	GitHub Actions: "pages build and deployment"	Runs by itself on every push to `main`. You can watch it in the repo's Actions tab. Once it failed with a GitHub

		server error (500); re-running fixed it.
Cache busting	`?v=14` on every asset link	Browsers keep old copies of CSS/JS. Raising the number makes it a new address, so everyone gets the fresh file at once.
Build step	`node tools/build-seo.mjs`	Generates the SEO parts from data. Safe to rerun; it replaces its own block instead of adding another.

Order matters. The domain switch was done so nothing ever pointed at an address that didn't work yet: DNS record first, wait until it resolved, then the GitHub custom domain, then HTTPS, then new links. GoatCounter counted both addresses during the switch, so no visits were lost.

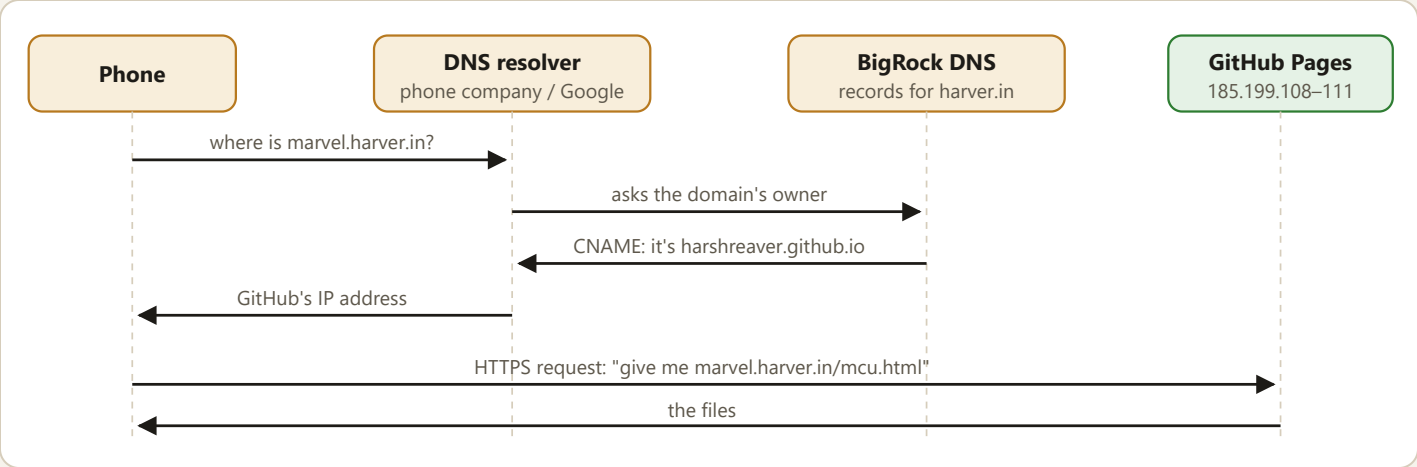
CHAPTER 4

Giving it a name

NETWORKING DEVOPS

> The site first lived at harshreaver.github.io/Marvel-Timelines. That's long, and the address you shared on WhatsApp. You already owned harver.in at BigRock, so the site moved to marvel.harver.in, a *subdomain* that costs nothing extra. Old links still work: GitHub forwards them.

What happens when someone types marvel.harver.in



DNS is the internet's phone book. The answer is cached for a while (the **TTL**), which is why DNS changes take minutes to hours to reach everyone. BigRock took about 2 hours here.

Records, explained

Record	Means	Used for
CNAME `marvel` → `harshreaver.github.io`	"This name is a nickname for that name"	Subdomains like marvel.harver.in. **This is what you have.**

A `@` → `185.199.108.153` (×4)	"This name lives at this IP address"	A bare domain like harver.info, if you ever host one on GitHub (plus a CNAME for `www`).

Setting it up again (step by step)

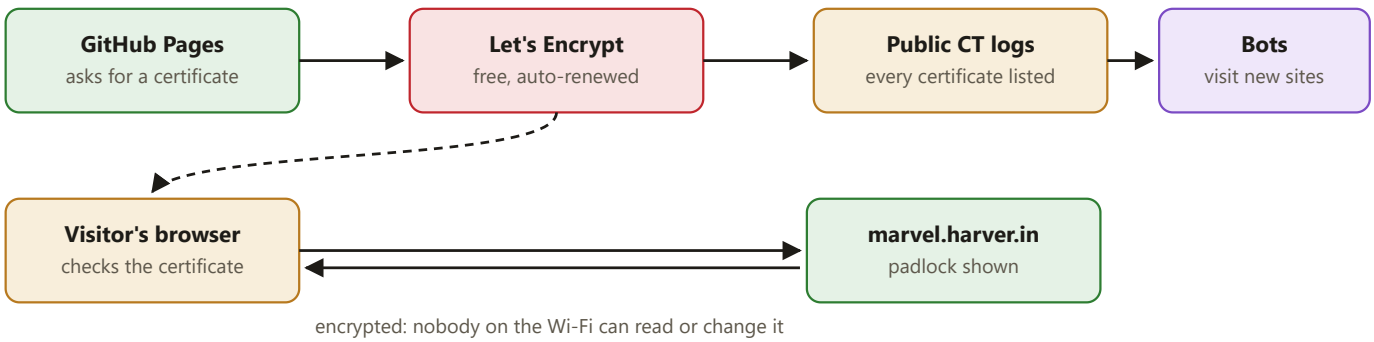
1. **BigRock:** Orders → harver.in → DNS Records → CNAME tab → Add CNAME Record. Name `marvel`, value `harshreaver.github.io`, save.
2. **Check:** in Command Prompt run `nslookup marvel.harver.in 8.8.8.8`. When it mentions github.io or 185.199..., it's live.
3. **GitHub:** repo → Settings → Pages → Custom domain → `marvel.harver.in` → Save. GitHub adds a `CNAME` file to the repo. Pull it in GitHub Desktop, and never delete it.
4. **HTTPS:** wait until "Enforce HTTPS" can be ticked (minutes to an hour), then tick it.

CHAPTER 5

Locking the door

SECURITY

> A site with no server and no database has very little to attack: there's no login to break into and no data to steal. Security work here was about the connection, the domain, and what other people's code can do on your pages.



HTTPS in one picture. The certificate log on the right explains the early visits from the US and Belarus: scanners watch those logs and visit brand-new sites within hours.

Topic	What's in place	Why it matters
HTTPS (TLS)	Let's Encrypt certificate, "Enforce HTTPS" ticked	Encrypts traffic and proves the site is really yours. http:// is forced to https://.
Subdomain takeover	CNAME record at BigRock + CNAME file on GitHub	If you ever delete the GitHub site or its custom domain, **also delete the CNAME record at BigRock**. A record that points at an unclaimed github.io site lets a stranger claim marvel.harver.in and show anything on it.
Third-party scripts	Only GoatCounter; images from Wikipedia/Wikimedia	Any script you load can do anything on your pages, so keep the list short and trusted.

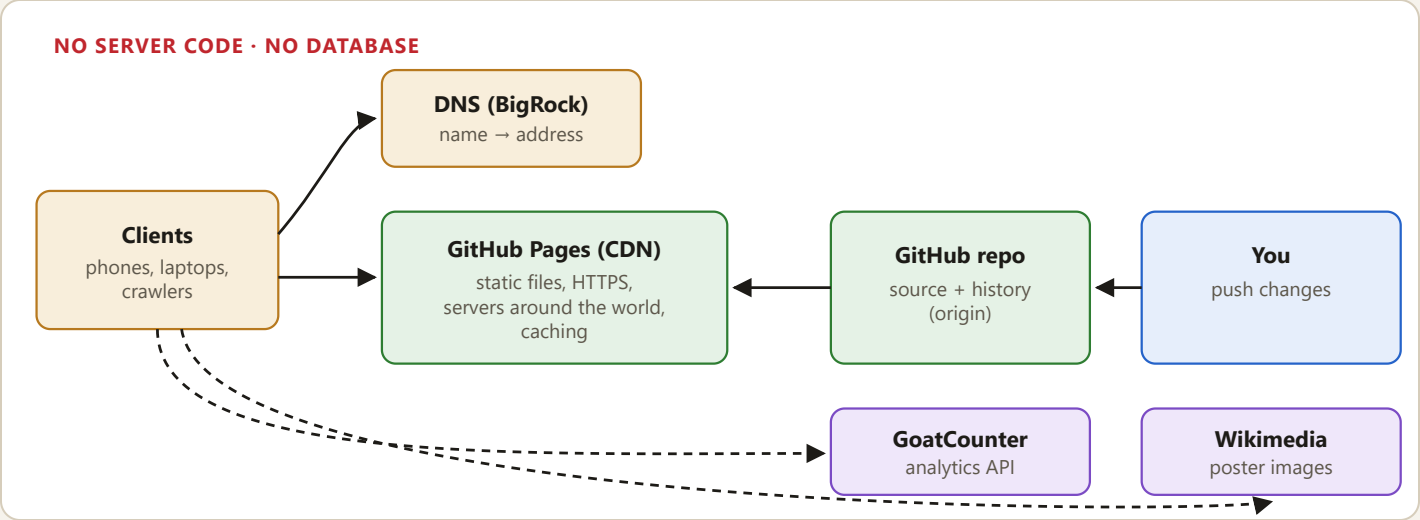
Secrets	None in the repo	The repo is public. Never commit passwords, API keys or tokens; anyone can read them, and bots scan GitHub for them.
Privacy	GoatCounter is cookieless; nothing personal stored	No cookie banner needed. Dark mode and section choices stay in the visitor's own browser.
Security headers	Not possible on GitHub Pages	SEO tools flag missing HSTS, CSP and X-Content-Type-Options. GitHub Pages doesn't let you set server headers, and with no logins or forms the risk is low. A host like Cloudflare Pages or Netlify would allow them.
Email on a public page	Address put together by app.js	Bots scan web pages for email addresses to spam. The address isn't written in the HTML; the script joins its two halves when the page loads, and most bots don't run scripts. It's also kept out of the README, because GitHub is scanned heavily.
Spam on the feedback form	A hidden "honeypot" box	People never see it, but spam bots fill in every box they find. If it's ticked, the message is quietly dropped. Messages also need some text, and an email address is checked before sending.
Verification file	`google1b416379081362bb.html`	Proves to Google you own the site. Harmless to keep public; deleting it loses Search Console access.

CHAPTER 6

Why it's built this way

HLD

> High-level design (HLD) is the view from above: which big pieces exist, how they talk, and why this shape was chosen over others. In interviews you'd be asked "design X", and you'd answer with a diagram like this, plus trade-offs.



The whole architecture. The dashed lines are calls the visitor's browser makes directly to other services.

The trade-offs

What this design wins

- **Cost:** free hosting, free HTTPS; only the domain costs money.
- **Scale:** a viral Reddit post is fine. GitHub's CDN serves the same files to everyone; there's no server to overload.
- **Speed:** files come from a nearby server; Rank Math measured 0.05–0.8 s responses.
- **Security:** no login, database or server code means very little to attack.
- **Simplicity:** deploying is just a push. Rolling back is a revert.

What it gives up

- **No accounts or saved data** across devices (watch lists would need a backend).
- **Crawlers that skip JavaScript** see less, which is why the plain lists and build script exist.
- **No control over server headers** (Chapter 5).
- **Depends on GitHub:** if Pages is down, so is the site. Limits: about 1 GB site size and 100 GB traffic a month, far above what you need.

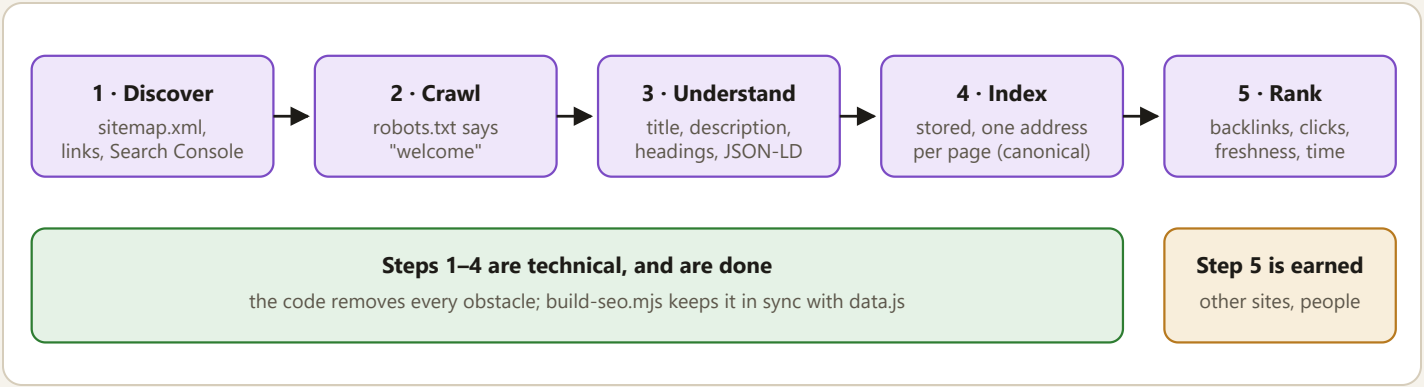
If it ever needs more. The usual next step is a small backend (for example Cloudflare Workers or Supabase) only for the parts that need it, such as user watch lists, while the pages stay static. That pattern is called **Jamstack**.

CHAPTER 7

Getting found

SEO WEB DEV

> A great site nobody can find gets no visitors. Search engines and AI assistants read a page's hidden labels first, and many don't run JavaScript. So each page got labels written for machines, a plain-text copy of its content, and a map of the site. Then free checkers were used to find the gaps.



The search pipeline. Code can take you through step 4; ranking (step 5) depends on other sites linking to you and real people clicking.

What each page carries

Label	Where you see it	Why it matters
Title, e.g. "MCU Timeline: Watch Order in Story & Release Order"	The blue link in Google, the browser tab	Matches what people type ("MCU watch order").
Meta description (121–147 characters)	The grey text under the link	Short enough not to be cut off; makes people click.
Headings: "X-MEN · Movies in order, both Fox timelines"	Big title and the small line under it	The H1 tells Google what the page is about.

Canonical link	Hidden	The one true address, so github.io copies don't compete.
Open Graph + `share.jpg`	The card when the link is shared on WhatsApp, LinkedIn, X	More taps on shared links.
JSON-LD structured data	Hidden	Every title as machine-readable facts (name, year, film or series, seasons, summary). AI systems can read it without running the site.
Plain title list, menu and footer links	Hidden in browsers (replaced instantly)	Crawlers without JavaScript see 700–6,500 words and 16–24 links per page instead of an empty shell.
`robots.txt`, `sitemap.xml`, `llms.txt`	/robots.txt etc.	Crawler rules, a list of all pages, and a plain summary some AI tools read.

Report card from the free checkers

Checker	Result	What's left, and why
Rank Math	79 → **81/100**	Minify CSS/JS (skipped to keep files editable), www redirect (doesn't apply to a subdomain), mobile speed (their test crashed).
Seobility	**69%**	Page structure and server both 100%; held back by "external factors" (3%), meaning backlinks.
SEORCE	Health **90**, domain authority **11**	The authority comes from harver.in (the old github.io address had 1). Most warnings are server headers GitHub doesn't allow, or false alarms.
Semrush, schema.org validator	0 errors	Structured data was fixed: Movie entries no longer carry episode counts.

CHAPTER 8

Watching who comes

DEVOPS SEO

> The site was shared on WhatsApp before stats were set up, so the first visits were only partly counted. Two dashboards now answer the two big questions: "who visited?" and "how did Google show us?"

GoatCounter: marvel-timelines.goatcounter.com

- Page views, countries, devices, where people came from.
- Unique visitors: the dashboard counts people, not every page load. For exact numbers, Settings → Export gives a CSV with a `Session` column (one per visitor).
- Its history can't be moved into Google Analytics: GA4 only accepts visits from the last few days, and it would need a cookie banner.
- Loaded by app.js only on the real addresses, so your local tests don't count.
- Ad blockers hide some visits, so real numbers are a bit higher.

Google Search Console: property `https://marvel.harver.in/`

- What people searched to find you, clicks, and which pages are indexed.
- Sitemap submitted; "Couldn't fetch" at first is normal.
- URL Inspection → Request indexing (about 10 a day).

Reading odd visits. Visits from the US or Belarus before anyone in those places knew about the site are usually bots (certificate-log scanners, SEO checkers, Google's own tester), or people on a VPN. Bot clues: Linux or headless Chrome, desktop screen sizes, one page, and a time matching when a checker ran. Your real audience so far is Android phones in India. In engineering this is called **observability**: being able to tell what's happening from the outside.

CHAPTER 9

Bugs we hit and what they taught

WEB DEV DEVOPS

> Every real project has bugs. What matters is finding the cause, not just hiding the symptom. These are the ones worth remembering, written the way engineers write a short post-mortem.

The old link that kept coming back

Symptom The footer still showed the old "Marvel-Timeline" link after it was fixed, and it led to a 404.

Cause Browsers kept the old app.js in their cache.

Fix `?v=` version tags on every asset.

Lesson Caching makes sites fast, so plan how to invalidate it.

The countdown that said "1 days"

Symptom Wrong wording, and no plan for after release.

Cause Time-of-day maths and a single hard-coded film.

Fix Whole calendar days in UTC, plural rules, and a list of films with "out" and "gone" dates.

Lesson Dates and time zones are a classic source of bugs. Test the edge days.

A duplicate SEO block

Symptom Running the build script twice doubled the tags.

Cause The clean-up search didn't match its own comment.

Fix Match everything from `seo:start` to `seo:end`, all occurrences.

Lesson Make scripts **idempotent**: running them twice gives the same result as once.

The About page with 143 items

Symptom A page meant to list nothing listed every title.

Cause In JavaScript an empty list `[]` counts as "true".

Fix Return `false` instead.

Lesson Know your language's truthy and falsy rules.

Invalid structured data

Symptom Semrush flagged 4 invalid items.

Cause Two specials were marked "Movie" but carried episode counts.

Fix Only TV series get seasons and episodes.

Lesson Validate generated data against its schema (validator.schema.org).

The deploy that failed for no reason

Symptom A push didn't go live.

Cause GitHub's own server error (500), not your files.

Fix Re-run, or push again.

Lesson Read the log before changing code. Not every failure is yours.

Borders cut off at 100% zoom

Symptom The top line of the section buttons vanished after the bar slid back.

Cause A CSS transform left the bar on half pixels, and the scrolling row clipped the border.

Fix Spare padding, and hiding the bar by moving its sticky position in whole pixels.

Lesson Bugs that show at only one zoom level are usually sub-pixel rounding.

Unreadable and mismatched colours

Symptom Mixed-colour section buttons; dark text on dark red; a white logo strip over the posters.

Cause Colours chosen per section, not per page theme.

Fix One page colour for all buttons, the strip removed (logos stay in the footer).

Lesson Users notice consistency. Check contrast in both themes.

CHAPTER 10

Everyday tasks, step by step

Add a new film or series

1. Open `assets/data.js`. The comment at the top explains every field. Copy a similar title's entry and change it. Entries are in release order.
2. If it's not out yet, add `up:1,rel:"2027-05-07"` (its release date). It drops the Upcoming tag on that day by itself. For an MCU title, also give it `so:` (its place in story order).
3. Poster: add an entry to `assets/media.js` with the Wikipedia image, or save a poster about 400px wide into `assets/posters/` and use `"img":"assets/posters/name.jpg","w":400,"h":600`.
4. If it belongs at a particular point in a story tree (X-Men forks, Spider-Verse lanes), it may also need adding to that page's `SPECS` in `assets/tree.js`.
5. Run `node tools/build-seo.mjs` (VS Code: Terminal → New Terminal).
6. Raise the version numbers (`data.js?v=`, `media.js?v=`) in all pages: Ctrl+Shift+H in VS Code replaces them everywhere.
7. Test locally, then commit and push.

Fix the character filter

If a title is missing from a character, or listed under the wrong one, open the `CHARACTERS` list at the bottom of `assets/data.js`. Add the title's `id` to that character's `add:[…]` list (missing) or `not:[…]` list (wrong). To add a new character, copy a line and change the name and pattern. Then raise `data.js?v=` in all pages.

Switch on the feedback form

1. Go to **web3forms.com**, enter the email that should receive feedback, and they send you an **access key** (no password needed).
2. Paste it between the quotes in `const FEEDBACK_KEY = "";` in `assets/app.js`.
3. Raise `app.js?v=` in all pages, push, and send yourself a test message from the live site.

The free plan allows 250 messages a month. Each one arrives as an email with the topic, rating, message, page and (if given) the visitor's email, so you can just press Reply.

Change the contact links

They're the `REPO_URL`, `LINKEDIN`, `INSTAGRAM` and `EMAIL` lines near the top of `assets/app.js`. Raise `app.js?v=` after changing.

After you watch Doomsday

Rewrite its `d:"…"` summary in data.js (it still describes the trailer and cast), then do steps 5 to 7 above.

Change the countdown

Edit the `FILMS` list at the top of `assets/home.js`: `out` is the release date, `gone` is when it hands over to the next film. Raise `home.js?v=` in index.html.

Change a page's colours

Each page's colours are one line near the top of `assets/style.css`, for example `[data-theme=tva]{--bg:…;--accent:…}`. Section button colours are the `.t-jump{--jump:…}` lines. Raise `style.css?v=` after changing.

Change a search title or description

Edit the `PAGES` list near the top of `tools/build-seo.mjs`, then run it. Keep titles under 60 characters and descriptions under 155.

CHAPTER 11

Getting traffic

> Nobody can promise a top Google spot or that Google's AI will recommend a site. The technical side is done. What decides it from here is other sites linking to yours, people clicking and staying, and time: a few weeks to appear for the site's own name, months for busy searches like "MCU watch order".

1. **Request indexing** for the main pages in Search Console (Chapter 8).
2. **Bing Webmaster Tools** (bing.com/webmasters): choose "Import from Google Search Console". Bing's index feeds ChatGPT search, Copilot and DuckDuckGo.
3. **Link from places you control:** LinkedIn (Featured, plus a short post about building it), the Website field on the GitHub repo, your GitHub profile README, harver.in.
4. **Answer where fans ask "what order should I watch":** Reddit (r/MCU, r/marvelstudios, r/xmen; read each subreddit's self-promotion rules first), Quora, a Letterboxd list. One good thread can beat months of search traffic, and each link raises your authority score.
5. **Stay fresh around releases:** Doomsday (18 December 2026), Beyond the Spider-Verse (June 2027), Secret Wars (December 2027). Update summaries soon after each.
6. **Watch what works:** GoatCounter for pages and sources, Search Console's Performance report for searches. Strengthen the pages that already get found.

CHAPTER 12

Which file does what

File	What it does	Edit by hand?
assets/data.js	Every title: name, year, summary, connections, release dates; character filter rules	Yes, most edits happen here
assets/media.js	Poster or title card, seasons and episodes per title	Yes, to add posters
assets/tree.js	Draws the vertical trees and the section bar (hide on scroll)	Only for tree layout
assets/app.js	Menu, dark theme button, footer and contact icons, feedback form, pop-ups, catalog and its filters, GoatCounter, release-date switch	Rarely
assets/home.js	Home page countdown and counts	For countdown dates
assets/style.css	All the looks: page themes, dark theme, phone layout	For design changes
*.html	The pages (a shell the scripts fill in, plus the dark-mode head script)	For header text, version numbers
multiverse.html	Also draws the lane map (its own script inside)	For map lanes and crossovers
tools/build-seo.mjs	Titles, descriptions, structured data, plain lists, sitemap, llms.txt	Only its PAGES list
sitemap.xml, llms.txt	Page lists for Google and AI tools	No, generated

CNAME	The custom domain	No, never delete
google1b416379081362bb.html	Proves to Google you own the site	No, never delete
robots.txt, favicon.png, apple-touch-icon.png, assets/share.jpg	Crawler rules, site icon, share preview image	Rarely

CHAPTER 13

Words to know

Term	Skill	Meaning, using this site as the example
Static site	HLD	Plain files served as they are; no server code or database. This whole site.
CDN	HLD	Servers around the world holding copies of your files, so each visitor gets them from nearby. GitHub Pages is one.
Client-side rendering	Web dev	The browser builds the page from data with JavaScript (tree.js).
Progressive enhancement	Web dev	Works as plain HTML first, gets richer with JavaScript (the static lists).
Design tokens	Web dev	Named values like `--accent` that a whole design is built from.
CI/CD	DevOps	Automatic build and deploy on every push (GitHub Actions).
Cache invalidation	DevOps	Making sure people get the new file, not an old copy (`?v=`).
Rollback	DevOps	Going back to a working version (revert a commit, push).
Idempotent	DevOps	Safe to run again and again with the same result (build-seo.mjs).
DNS, CNAME, A record, TTL	Networking	The internet's phone book, a nickname record, an address record, and how long answers are cached.
TLS / HTTPS, certificate	Security	Encrypted connection, and the digital ID that proves the site is yours (Let's Encrypt).

Subdomain takeover	Security	A stranger claiming your name because DNS still points at a service you stopped using.
Attack surface	Security	Everything an attacker could poke at. Tiny here: no logins, forms or database.
Separation of concerns	LLD	Each module has one job (data vs drawing vs styling).
Single source of truth	LLD	One place holds each fact (data.js); everything else reads from it.
Canonical URL	SEO	The one official address of a page.
Structured data (JSON-LD)	SEO	Facts in a machine-readable format search engines and AI understand.
Backlinks, domain authority	SEO	Links from other sites, and a score of how trusted a domain is (11 for marvel.harver.in).
Observability	DevOps	Seeing what's happening in a live system (GoatCounter, Search Console, Actions logs).
Post-mortem	All	A short write-up of a bug: symptom, cause, fix, lesson (Chapter 9).